

Setup Python - pyenv, pipx, uv i Poetry

2026-08-17

Data Engineering Course

■ Setup Python

Przygotowanie srodowiska do lekcji Python

```
system -> pyenv -> Python -> pipx/uv -> Poetry ->  
.venv projektu
```

Cel: kazdy uczestnik ma uruchamiac ten sam projekt na tej samej wersji Pythona.

■ Po co setup

Bez uporządkowanego środowiska każdy może mieć:

```
inna wersje Pythona,  
inne paczki,  
inny interpreter,  
inny folder virtualenv,  
inne błędy przy uruchomieniu.
```

W Data Engineeringu to szybko psuje powtarzalność projektu.

```
pyenv
  wybiera wersje Pythona

pipx
  instaluje globalne narzedzia CLI w izolacji

uv
  szybkie install/run/tools dla Pythona

Poetry
  zaleznosci projektu, pyproject.toml,
  poetry.lock, .venv
```

Jedno narzedzie = jedna odpowiedzialnosc.

■ Docelowa struktura

Po setupie projekt ma mieć:

```
homework/lesson_05/
├── .python-version
├── pyproject.toml
├── poetry.lock
├── .venv/                                # lokalnie, nie
commitujemy
├── python_basics.py
├── test_python_basics.py
└── orders.csv
```

Do review commitujesz kod i pliki konfiguracyjne.

Nie commitujesz `.venv/`.

■ Linux / WSL: system packages

Ubuntu / WSL Ubuntu:

```
sudo apt update
sudo apt install -y make build-essential
libssl-dev zlib1g-dev \
    libbz2-dev libreadline-dev libsqlite3-dev curl
git libffi-dev
```

Te paczki sa potrzebne, zeby **pyenv** mogl zbudowac Pythona lokalnie.

Instalacja:

```
curl https://pyenv.run | bash
```

Dodaj do `~/.zshrc`:

```
export PYENV_ROOT="$HOME/.pyenv"  
[[ -d $PYENV_ROOT/bin ]] && export PATH="$  
PYENV_ROOT/bin:$PATH"  
eval "$(pyenv init -)"
```

Potem uruchom nowy shell.

```
pyenv --version  
pyenv install 3.13.0  
pyenv global 3.13.0  
python --version
```

Oczekiwane:

```
Python 3.13.0
```

pyenv global ustawia domyslna wersje dla terminala.

■ pyenv local

W katalogu projektu:

```
cd homework/lesson_05
pyenv local 3.13.0
cat .python-version
python --version
```

`.python-version` jest informacja dla `pyenv`:

w tym folderze użyj tej wersji Pythona

■ pipx

Instalacja:

```
python -m pip install --user pipx  
python -m pipx ensurepath  
exec zsh  
pipx --version
```

Do czego używamy **pipx**?

Do instalacji narzędzi CLI, np. Poetry.

pipx izoluje narzędzia od projektów.

Instalacja Linux / WSL:

```
curl -LsSf https://astral.sh/uv/install.sh | sh  
uv --version
```

Przykłady użycia:

```
uv tool install ruff  
uvx ruff --version  
uv run --with pydantic python script.py
```

 jest szybkie i przydatne do jednorazowych runów.

■ Poetry

Rekomendacja dla tej lekcji: instaluj Poetry przez `pipx`.

```
pipx install poetry  
poetry --version  
poetry config virtualenvs.in-project true
```

Dlaczego `virtualenvs.in-project true`?

```
.venv powstaje w katalogu projektu,  
łatwiej zobaczyc, ktore srodowisko nalezy do  
projektu,  
łatwiej debugowac interpreter.
```

■ Projekt Poetry

W katalogu homeworku:

```
mkdir -p homework/lesson_05
cd homework/lesson_05
pyenv local 3.13.0
poetry init --name lesson-05-python-intro --python
"^3.13" --no-interaction
poetry add --group dev pytest
```

Po tym powinny powstac:

```
.python-version
pyproject.toml
poetry.lock
.venv/
```

■ Uruchamianie projektu

Z katalogu `homework/lesson_05`:

```
poetry run python python_basics.py  
poetry run pytest
```

Najwazniejsza zasada:

```
python           moze wskazywac przypadkowy  
interpreter  
poetry run python  uzywa interpretera z .venv tego  
projektu
```

Przez Homebrew:

```
brew install pyenv pipx uv
pipx ensurepath
pipx install poetry
poetry config virtualenvs.in-project true
pyenv install 3.13.0
pyenv global 3.13.0
```

Jesli shell nie widzi `pyenv`, sprawdź wpisy w pliku startowym `shell'a`.

Rekomendacja kursowa:

Windows + WSL Ubuntu

Wtedy użyj instrukcji Linux / WSL.

Natywnie w PowerShell:

```
winget install pyenv-win.pyenv-win  
python -m pip install --user pipx  
python -m pipx ensurepath  
powershell -ExecutionPolicy ByPass -c "irm  
https://astral.sh/uv/install.ps1 | iex"  
pipx install poetry
```

■ Verification checklist

Po setupie sprawdź:

```
python --version
pyenv --version
pipx --version
uv --version
poetry --version
poetry env info
```

W katalogu projektu:

```
cat .python-version
ls -la .venv
poetry run python --version
poetry run pytest
```

■ Najczęstsze problemy

python ma zła wersję

-> sprawdź pyenv local/global i .python-version

poetry nie tworzy .venv w projekcie

-> poetry config virtualenvs.in-project true

command not found: pyenv/pipx/uv/poetry

-> otwórz nowy shell albo sprawdź PATH

pytest nie działa

-> poetry add --group dev pytest

■ Co commitować

Commitujesz:

```
.python-version  
pyproject.toml  
poetry.lock  
python_basics.py  
test_python_basics.py  
orders.csv  
notes.md
```

Nie commitujesz:

```
.venv/  
__pycache__/  
.pytest_cache/  
.coverage
```

■ Closing check

Powiedz na głos:

```
pyenv wybiera wersje Pythona.  
pipx instaluje globalne narzędzia CLI.  
uv pomaga szybko uruchamiać i instalować  
narzędzia.  
Poetry zarządza zależnościami projektu.  
poetry run uruchamia kod w .venv projektu.
```